

Ejercicios de TypeScript — Sesiones 2 y 3

🚀 **Ruta React + Tauri.** Todo el catálogo está orientado a TypeScript aplicado a React y a Tauri. Los temas ajenos a ese objetivo (enums, clases y herencia, `.d.ts`, manipulación manual del DOM y Web APIs del navegador) se han **eliminado** del catálogo: React se encarga del DOM y la app es TypeScript *erasable-only*.

✅ **Soluciones.** Cada bloque resuelto enlaza a su fichero en [ejercicios/soluciones/](#). Hoy están **fuera del repositorio** (`.gitignore`): al publicarlos, los enlaces se activarán automáticamente. Solo los bloques de teoría (S2·1, S2·2 y S3·10) no tienen fichero de solución propio. 🧪 **Ejercicio ejecutable** en [ejercicios/s0X/](#) · 📖 **Teoría** en [apuntes/s0X/](#).

🎯 Prioridad para React y Tauri (P1–P5)

Escala afinada: dentro de lo imprescindible se distingue el **núcleo (P1)** de lo **necesario pero complementario (P2)**. Mide la importancia cara a *usarlo luego en React y Tauri*; por **decisión pedagógica**, **S2·13 Funciones es P1** (queremos que se trabaje mucho), aunque su peso conceptual en React/Tauri sea menor que el del resto del núcleo.

Prioridad	Significado
P1 · Núcleo imprescindible	No se escribe ni un componente típico sin esto.
P2 · Necesario pero complementario	Imprescindible a medio plazo; completa el núcleo.
P3 · Uso habitual	Aparece a diario; se interioriza en las sesiones React/Tauri.
P4 · Útil pero dilatable	Según caso; no bloquea.
P5 · Prescindible	Se omite sin problema (p. ej. overloads, métodos ES2023/ES2025).

Resumen sobre los **26 bloques con ejercicio**: **P1 = 10 · P2 = 6 · P3 = 6 · P4 = 4 · P5 reservado** (la teoría S2·1/S2·2 y S3·10 queda fuera, solo lectura/config).

Sesión 2

Bloque	Ejercicio	Prio	Por qué (React/Tauri)
3 · Tipos Primitivos	s02/01	P2	Base de todo el tipado, pero elemental y breve.
4 · Type Inference	s02/01 · s02/03	P2	El editor deduce tipos (inferencia contextual en map , hooks).
5 · Tipos Especiales	s02/03	P2	unknown / never esenciales; se aplican de lleno con los guards (P1).
6 · Union Types e Intersección · 15 ·	s02/04	P1	Props multiforma, estado discriminado y discriminated unions (estados de una

Bloque	Ejercicio	Prio	Por qué (React/Tauri)
Narrowing			petición/evento).
7 · Interfaces · 8 · Type Aliases	s02/05	P1	<code>interface Props</code> = contrato de cada componente; modelas entidades.
9 · Conversión de tipos · 12 · Operadores	s02/10	P3	JSON en persistencia y <code>??</code> para defaults; ternario/short-circuit a diario.
10 · Type Assertions	s02/09	P3	<code>as/as const</code> con <code>invoke</code> , refs y constantes.
11 · Type Guards Avanzados	s02/07	P1	Validar lo que llega de <code>invoke</code> /JSON: la regla del curso (nunca <code>any</code>).
13 · Funciones en Profundidad	s02/06	P1 (pedagógica)	Handlers <code>onClick</code> , callbacks y la semilla de <code>useState<T></code> ; <code>overloads</code> → P5.
14 · Estructuras de control de flujo	s02/11	P2	<code>switch+never</code> = el patrón <code>useReducer</code> /estados ok-cargando-error (React II).
16 · Ámbito (Scope) y closures	s02/08	P4	Explica los hooks por dentro; pocos closures complejos en componentes.
🧩 Reto final de S2	s02/12	P4	Integración/repaso de S2; sin concepto nuevo.

Sesión 3

Bloque	Ejercicio	Prio	Por qué (React/Tauri)
1 · Arrays y Tuplas	s03/01	P2	La tupla <code>[valor, setter]</code> de <code>useState</code> y las listas básicas.
2 · Arrays: métodos fundamentales	s03/06	P1	<code>map/filter/reduce/find</code> omnipresentes en el JSX de las listas.
3 · Set	s03/06	P3	Deduplicar listas y cachés; casos concretos.
4 · Map	s03/06	P3	Caché/persistencia clave-valor; <code>Record</code> cubre la mayoría.
5 · Objetos en profundidad	s03/07	P2	Entidades, formularios, JSON y cloning sin mutar el estado.
6 · <code>keyof</code> , <code>typeof</code> y <code>satisfies</code>	s03/04	P4	Formularios tipados y configs; no bloquea (⚡ opcional).
7 · Generics	s03/02	P1	<code>useState<T></code> , <code>invoke<T></code> y componentes <code><T></code> (tabla genérica).
8 · Utility Types	s03/03	P3	<code>Partial/Pick/Omit/Record</code> en formularios de edición y props.

Bloque	Ejercicio	Prio	Por qué (React/Tauri)
9 · Módulos en TypeScript	s03/05/	P4	Se aprende con los repos; <code>import type</code> al tipar <code>invoke</code> .
10 · Strict Mode y Configuración	—	(teoría)	Config de <code>tsconfig</code> ; no hay ejercicio.
11 · Programación asíncrona	s03/08	P1	<code>Promise.all</code> + <code>try/catch</code> = patrón <code>invoke</code> y del <code>useEffect</code> de carga.
12 · Desestructuración/spread/optional chaining	s03/09	P1	El puente: destructuring de props, spread de <code>setState</code> , <code>?./??</code> — lo primero de cada componente.

Sesión 2: Introducción a TypeScript (Parte 1)

1. ¿Qué es TypeScript?

 **Teoría:** `sesion00.md` (fundamentos, ecosistema, Tauri vs Electron).

1. Verifica en tu terminal que `node`, `npm`, `rustc` y `cargo` están instalados ejecutando `--version` en cada uno. Anota las versiones.
2. Crea un proyecto web mínimo con Vite: `npm create vite@latest mi-prueba -- --template react-ts`. Abre `src/App.tsx` y busca una anotación de tipo (p. ej. `useState<number>(0)`). ¿Dónde aparece el `: number` implícito?
3. Navega a la carpeta del proyecto generado y abre `package.json`. Identifica qué dependencias van en `dependencies` y cuáles en `devDependencies`. ¿Por qué `typescript` y `vite` están en `devDependencies` y no en `dependencies`?
4. ¿Qué diferencia clave de arquitectura hay entre Tauri y Electron en cuanto al backend? (Respuesta basada en la tabla de la sesión 00).

2. Instalación y Configuración Básica

 **Teoría:** `sesion01.md` (instalación por SO, creación de proyecto Tauri, terminología de comandos).

La configuración de `tsconfig.json` ya está lista en `ejercicios/tsconfig.json` (strict, erasable-only). Los ejemplos ejecutables de la teoría están incrustados en los apuntes como bloques "📦 Ejemplo completo"; para correrlos, cópialos a `ejercicios/tmp/<nombre>.ts` y ejecuta `npm run tsx tmp/<nombre>.ts`.

1. Crea un proyecto Tauri de prueba: ejecuta `npm create tauri-app@latest mi-tauri-test` (elige React + TypeScript + npm). Ejecuta `npm install` y luego `npm run tauri dev`. Comprueba que se abre una ventana nativa con el frontend de React.
2. Una vez creado, explora la estructura de carpetas del proyecto. Nombra al menos 4 archivos/carpetas clave dentro de `src-tauri/` y explica para qué sirve cada uno (usa la guía de scaffolding como referencia).
3. En el archivo `src-tauri/src/lib.rs`, localiza la función marcada con `#[tauri::command]`. ¿Qué hace? Ahora abre `src/App.tsx` y busca la llamada `invoke(...)`. ¿Qué le pasa como argumento y qué devuelve?

4. Verifica los comandos básicos de la terminal del anexo: ejecuta `npm --version`, `npx --version`, `cargo --version` y `rustc --version`. ¿Cuál de estos comandos pertenece al ecosistema de Node y cuál al de Rust?

3. Tipos Primitivos

 **Ejercicio:** `s02/01-tipos-primitivos.ts`.  **Solución:** `soluciones/s02/01-tipos-primitivos.ts`.  Prioridad **P2** — base de todo el tipado, pero elemental y breve.

1. Declara variables de tipo `string`, `number`, `boolean`, `null` y `undefined`, asígnales valores e imprímelas.
2. Intenta reasignar un `number` a una variable declarada como `string`. ¿Qué error obtienes?
3. Convierte un `number` a `string`, `boolean` y `number` de nuevo con `String()`, `Boolean()` y `Number()`.
4. Declara variables con tipo explícito: tu nombre (`string`), tu edad (`number`), un booleano que indique si estás aprendiendo TypeScript y un arreglo de números con tus 3 números favoritos.
5. Declara `precio: number = 99.99` con tipado explícito y `activo` con `true` dejando que TypeScript infiera el tipo.
6. Crea `nombre` y `apellido` (ambos `string`) y combínalos con una template literal en `nombreCompleto`.
7. Escribe `longitudTexto(numero: number): number` que devuelva cuántos dígitos tiene el número (pista: conviértelo a `string` primero).

4. Type Inference

 Cubierto dentro de `s02/01-tipos-primitivos.ts` (apartado 2) y `s02/03-tipos-especiales.ts`.  **Solución:** `soluciones/s02/01-tipos-primitivos.ts` y `soluciones/s02/03-tipos-especiales.ts`.  Prioridad **P2** — inferencia contextual (`map`, `hooks`): el editor deduce los tipos por ti.

1. Declara variables sin anotación de tipo y observa los tipos inferidos por el editor.
2. Crea un array con números y strings mezclados. ¿Qué tipo infiere TypeScript?
3. Declara una `const` vs `let` con el mismo valor literal. Compara los tipos inferidos.
4. Usa inferencia contextual con `map` — ¿cómo sabe TypeScript el tipo del parámetro?

5. Tipos Especiales: any, unknown, never, void

 **Ejercicio:** `s02/03-tipos-especiales.ts`.  **Solución:** `soluciones/s02/03-tipos-especiales.ts`.  Prioridad **P2** — `unknown/never` son esenciales; se aplican de lleno con los guards (P1).

1. Declara una variable `any` y asígnale distintos tipos. Llama a un método inexistente — ¿qué ocurre en compilación y en runtime?
2. Repite el paso anterior con `unknown`. ¿Qué necesitas hacer para poder llamar a un método sobre `unknown`?
3. Escribe una función que devuelva `void` y otra que devuelva `never` (que lance un error).
4. Implementa un switch exhaustivo con un tipo unión y una variable `never` en el `default`.
5. Escribe `logMensaje(texto: string): void` que imprima el mensaje con `console.log`.
6. Crea `procesarDesconocido(valor: unknown)`: si es `string`, imprime su longitud; si no, imprime "No es un texto".
7. Escribe `lanzarError(mensaje: string): never` que lance `new Error(mensaje)`.
8. Refactoriza esta función para que no use `any`, comprobando que `val` sea un número antes de operar:

```
function duplicar(val: any) { return val * 2; }
```

9. Dado `type Forma = "circulo" | "cuadrado"`, escribe una función con `switch` cuyo `default` asigne el parámetro a una variable `never` para garantizar el exhaustive checking.

6. Union Types e Intersección de Tipos

 **Ejercicio:** `s02/04-unions-narrowing.ts`.  **Solución:** `soluciones/s02/04-unions-narrowing.ts`.  Prioridad **P1** — props multiforma y estado discriminado: el pan de cada día de un componente.

1. Define un tipo `ID` que sea `string | number`. Crea una función que acepte `ID` y devuelva su longitud como `string`.
2. Crea dos interfaces `Persona` y `Empleado`. Combínalas con intersección `&` y crea un objeto que cumpla ambas.
3. Haz una función que acepte `string | string[]` y devuelva el primer elemento.

7. Interfaces

 **Ejercicio:** `s02/05-interfaces-types.ts`.  **Solución:** `soluciones/s02/05-interfaces-types.ts`.  Prioridad **P1** — `interface Props` es el contrato de cada componente.

1. Define una interface `Cancion` con `titulo`, `artista`, `duracion` (opcional) y `reproducir()` (método).
2. Extiende `Cancion` con `Podcast` que añada `episodio` y `descripcion`.
3. Crea dos interfaces con el mismo nombre y observa el declaration merging.
4. Añade una index signature a una interface `Diccionario` con clave `string` y valor `string`.
5. Crea una interface `Producto` con `id` y `precio` (obligatorios) y `descuento` (opcional). Declara una variable `laptop` que la cumpla.
6. Crea una interface `ConfiguracionServidor` con `puerto` y `host` marcadas como `readonly` para que no se puedan modificar tras crear el objeto.

8. Type Aliases

 **Ejercicio:** `s02/05-interfaces-types.ts`.  **Solución:** `soluciones/s02/05-interfaces-types.ts`.  Prioridad **P1** — modelar entidades y tipos de datos de la app.

1. Define un type alias `Coordenadas` como `{ x: number; y: number }`. Crea una función que lo acepte.
2. Crea un type `Resultado<T>` genérico con `exito: boolean`, `datos: T`.
3. Define un type para un callback `(err: Error | null, data?: unknown) => void`.
4. Compara: ¿cuándo usarías `type` vs `interface`?

9. Conversión de tipos

 **Ejercicio:** `s02/10-conversion-operadores.ts` (`JSON`, `??`, `||/??`) — la base de `String/Number/Boolean` está en `s02/01-tipos-primitivos.ts`.  **Solución:** `soluciones/s02/10-conversion-operadores.ts` (`JSON`, `??`, `||/??`) — la base de `String/Number/Boolean` está en `soluciones/s02/01-tipos-primitivos.ts`.  Prioridad **P3** — `JSON` en persistencia y `??` para defaults; se interioriza con Tauri/formularios.

1. Convierte explícitamente: `number` → `string`, `string` → `number`, `string` → `boolean`.
2. Usa `JSON.stringify` y `JSON.parse` con una interface tipada.
3. Escribe una función que use el operador `??` para proporcionar valores por defecto.
4. Diferencia entre `||` y `??` con un ejemplo donde `0` sea un valor válido.

10. Type Assertions

 **Ejercicio:** `s02/09-asepciones.ts`.  **Solución:** `soluciones/s02/09-asepciones.ts`. 

Prioridad **P3** — `as/as` `const` con `invoke`, refs y constantes; lo crítico ya lo cubren los guards.

1. Usa `as` para estrechar un valor `unknown` procedente de `JSON.parse` (una API / una llamada que devuelve JSON) y conviértelo a la interface `Película`. ¿Qué ganancia de tipo aporta `as`? ¿Qué riesgo tiene?
2. Prueba el non-null assertion `!` en una propiedad opcional de un objeto.
3. Crea un objeto y un array `as const` e intenta modificar una propiedad. ¿Qué error obtienes?
4. En React: estrecha la forma mínima de un evento tipado como `unknown` con `as`.
5. Deriva el tipo de un elemento de un array con `typeof arr[number]` a partir de un `as const`.
6. Diferencia entre `as` y `<tipo>`. ¿Cuándo no funciona la segunda sintaxis?

11. Type Guards Avanzados

 **Ejercicio:** `s02/07-type-guards.ts`.  **Solución:** `soluciones/s02/07-type-guards.ts`. 

Prioridad **P1** — validar lo que llega de `invoke`/JSON: la regla del curso (nunca `any`).

1. Crea un custom type guard `esAdmin(usuario)` que compruebe si un usuario tiene rol `"admin"`.
2. Usa un type predicate en un `filter` para obtener solo los elementos de un tipo concreto de una unión.
3. Implementa una assertion function `asegurarNumero(valor: unknown): asserts valor is number`.

12. Operadores

 **Ejercicio:** `s02/10-conversion-operadores.ts`.  **Solución:** `soluciones/s02/10-conversion-operadores.ts`.  Prioridad **P3** — ternario/`??`/short-circuit: se usan a diario y se aprenden rápido.

1. Compara `==` vs `===` con `5` y `"5"`. ¿Qué resultados obtienes?
2. Usa el operador ternario para asignar un valor según una condición.
3. Escribe una función que use short-circuit evaluation para evitar una llamada si una condición es falsa.

13. Funciones en Profundidad

 **Ejercicio:** `s02/06-funciones.ts`.  **Solución:** `soluciones/s02/06-funciones.ts`. 

Prioridad **P1** (*decisión pedagógica*) — handlers `onClick`, callbacks y la semilla de `useState<T>`; *overloads* → P5.

1. Escribe una función con parámetro obligatorio, otro opcional y otro con valor por defecto.
2. Crea una función con rest parameters que sume todos los números recibidos.
3. Escribe `calcularTotal(precioBase, impuesto = 0.21)` que devuelva el total con impuestos.
4. Implementa un closure `crearContador` que devuelva `incrementar`, `decrementar` y `valor`.

5. Escribe function overloads para `formatearEntrada` que acepte `string` o `number[]` y devuelva el tipo correspondiente.
6. Asigna una función a una variable (expresión funcional): `const cuadrado: (n: number) => number = (n) => n * n`; y úsala. Prueba también una función flecha que no use el parámetro explícito devuelto por el tipo.
7. Declara un tipo `Operacion = (a: number, b: number) => number` y crea funciones `sumar`, `restar` y `multiplicar` que la cumplan. Guarda las tres en un array y recórrelas.

14. Estructuras de control de flujo

 **Ejercicio:** `s02/11-control-de-flujo.ts` (switch exhaustivo con `never` = patrón reducir de React).  **Solución:** `soluciones/s02/11-control-de-flujo.ts` (switch exhaustivo con `never` = patrón reducir de React).  Prioridad **P2** — `switch+never` = el patrón `useReducer`/estados `ok·cargando·error` (llega en React II).

1. Escribe un `if/else if/else` que clasifique una nota numérica en Sobresaliente, Notable, Aprobado, Suspenso.
2. Crea un `switch` exhaustivo con un tipo unión de 4 valores literales. Incluye exhaustiveness check con `never`.
3. Recorre un array de strings con `for...of` y suma sus longitudes.
4. Genera la secuencia `hasta...1` en un array usando `for...of` sobre un rango (`Array.from`).

15. Literal Types y Type Narrowing

 **Ejercicio:** `s02/04-unions-narrowing.ts`.  **Solución:** `soluciones/s02/04-unions-narrowing.ts`.  Prioridad **P1** — `typeof/in` y discriminated unions: los estados de una petición o evento.

1. Define un tipo literal `Direccion` con valores `"N" | "S" | "E" | "O"`. Escribe una función que devuelva el nombre completo.
2. Crea una discriminated union `Triangulo | Cuadrado` con la propiedad discriminante `tipo`. Implementa `calcularArea`.
3. Usa `typeof` narrowing en una función que acepte `string | number | boolean` y aplique una transformación distinta a cada caso.
4. Usa `in` narrowing para distinguir entre dos interfaces.
5. Define un type alias `EstadoPedido` con las literales `"pendiente" | "enviado" | "entregado"` y crea `actualizarEstado(estado)` que imprima en consola la notificación del cambio.
6. Dado `const roles = { admin: "ADMINISTRADOR", user: "USUARIO_ESTANDAR" }` as `const`, deriva un tipo a partir de los valores de `roles` que solo admita `"ADMINISTRADOR" | "USUARIO_ESTANDAR"`.
7. Escribe `procesarEntrada(entrada: string | number)`: si es `string` devuelve el texto en mayúsculas; si es `number`, el doble del número.

16. Ámbito (Scope) y closures

 **Ejercicio:** `s02/08-scope-closures.ts`.  **Solución:** `soluciones/s02/08-scope-closures.ts`.  Prioridad **P4** — explica los hooks por dentro; pocos closures complejos en componentes.

1. Razona qué imprime cada `console.log` de un bloque con variables del mismo nombre en distintos ámbitos.
2. Crea un closure `crearContador` que devuelva funciones `incrementar`, `decrementar` y `valor` con estado independiente por instancia.
3. Crea `hacerSaludos(nombres)` que devuelva un array de funciones que saluden al nombre i-ésimo.
4. Crea `memoizar(fn)` que guarde resultados en un `Map` para argumentos repetidos.

🎯 Reto final de S2

🔧 **Ejercicio:** `s02/12-retro-s02.ts` · ✅ **Solución:** `soluciones/s02/12-retro-s02.ts`. Integra TODO lo visto en S2 en un único programa: uniones de literales, narrowing, *type guards* con predicados, parámetros por defecto, ternario/??, `for...of`, `switch` exhaustivo con `never`, closures y aserciones. La entrada en React: piensa en ello como el esqueleto de un `useReducer`. 🎯 Prioridad **P4** — integración/repaso de S2; sin concepto nuevo para React/Tauri.

Sesión 3: Introducción a TypeScript (Parte 2)

1. Arrays y Tuplas

🔧 **Ejercicio:** `s03/01-arrays-tuplas.ts` · ✅ **Solución:** `soluciones/s03/01-arrays-tuplas.ts`. 🎯 Prioridad **P2** — la tupla `[valor, setter]` de `useState` y las listas básicas.

1. Crea un array de números, otro de strings y otro mixto `(string | number)[*]`.
2. Define una tupla `[string, number, boolean]` con datos de un usuario. Desestructúrala.
3. Crea una tupla etiquetada `[lat: number, lng: number]` y un array `readonly` de números. Intenta modificar ambos y observa los errores.
4. Escribe una función que reciba un `readonlyArray<number>` y devuelva su suma.
5. Define una tupla `Coordenadas2D` de exactamente dos números `[x, y]` y crea `moverPunto(tupla, dx, dy)` que devuelva una nueva tupla con la posición actualizada.
6. Declara un array de strings con la sintaxis genérica `Array<string>` e inicialízalo con tres ciudades.
7. Crea un array `respuestas` que contenga solo `boolean | number` y rellénalo con al menos 4 elementos variados.
8. Define una tupla `RespuestaHTTP` que contenga un código de estado (`number`, obligatorio) y un mensaje de error (`string`, opcional).
9. Crea una tupla `readonly [number, number]` para un punto cartesiano e intenta modificarla por asignación directa. ¿Qué ocurre?
10. Dada `const usuario: [number, string, boolean] = [1, "Ana", true]`, desestructúrala en variables individuales tipadas explícitamente.

2. Arrays: métodos fundamentales

🔧 **Ejercicio:** `s03/06-arrays-set-map.ts` · ✅ **Solución:** `soluciones/s03/06-arrays-set-map.ts`. 🎯 Prioridad **P1** — `map/filter/reduce/find` son omnipresentes en el JSX de las listas.

1. Dado `[1, 2, 3, 4, 5]`, usa `map` para duplicar, `filter` para pares, `reduce` para sumar.
2. Usa `find`, `findIndex`, `some` y `every` sobre un array de objetos `Producto`.
3. Usa `toSorted`, `toReversed` y `with` (ES2023) y comprueba que el original no muta.
4. Implementa una búsqueda binaria tipada.

5. Dada la interface `Usuario { nombre: string; activo: boolean }`, escribe `obtenerActivos(usuarios: Usuario[])` que devuelva un nuevo array solo con los `activo === true`.
6. Usa los métodos que **mutan** el array en su lugar: `push` y `pop` sobre una pila, y `sort` con un comparador numérico `(a, b) => a - b`. Comprueba que `sort` modifica el array original.
7. Crea una copia de un array con `slice()` y con el spread `[...arr]`, y modifica la copia sin afectar al original (contrasta con `sort` que sí muta).

3. Set: conjunto de valores únicos

 **Ejercicio:** `s03/06-arrays-set-map.ts`.  **Solución:** `soluciones/s03/06-arrays-set-map.ts`.
 Prioridad **P3** — deduplicar listas y cachés; casos concretos.

1. Crea un `Set<string>` con nombres de colores. Añade, elimina y comprueba existencia.
2. Dado un array con duplicados, elimínalos usando `Set`.
3. Dados dos conjuntos `A` y `B`, calcula: unión, intersección y diferencia.
4. Prueba los métodos nativos ES2025: `union`, `intersection`, `difference`, `isSubsetOf`.

4. Map: diccionario clave-valor

 **Ejercicio:** `s03/06-arrays-set-map.ts`.  **Solución:** `soluciones/s03/06-arrays-set-map.ts`.
 Prioridad **P3** — caché/persistencia clave-valor; `Record` cubre la mayoría de casos.

1. Crea un `Map<string, number>` con nombres de personas y sus edades. Itera sobre él.
2. Implementa una caché simple con `Map<string, { data: unknown; timestamp: number }>`.

5. Objetos en profundidad

 **Ejercicio:** `s03/07-objetos.ts`.  **Solución:** `soluciones/s03/07-objetos.ts`.  Prioridad **P2** — entidades, formularios, JSON y cloning sin mutar el estado.

1. Dado un objeto `persona`, usa `Object.keys`, `Object.values` y `Object.entries`.
2. Usa destructuring avanzado: alias, anidado y valores por defecto.
3. Clona un objeto con `structuredClone` — ¿qué tipos preserva que `JSON.parse(JSON.stringify(x))` no?
4. Usa `Object.groupBy` para agrupar productos por categoría.
5. Usa `Object.freeze` y comprueba que el objeto es readonly en runtime.

6. keyof, typeof y satisfies

 **Opcional** (profundización de TS). En la ruta React + Tauri es raro tener que usarlo a diario; pero `keyof` asoma en funcionalidades tipadas (claves de formularios, `Object.keys` as `Array<keyof T>`) y `satisfies` en configuraciones. Prioriza el 4 y 5 antes que este.  **Ejercicio:** `s03/04-keyof-typeof-satisfies.ts`.  **Solución:** `soluciones/s03/04-keyof-typeof-satisfies.ts`.  Prioridad **P4** — `keyof` en formularios tipados y `satisfies` en configs; no bloquea (⚡ opcional).

1. Dada una interface `Usuario { id: number; nombre: string; email: string }`, usa `keyof` para obtener sus claves como tipo.
2. Crea un objeto `config` con `url`, `port`, `timeout` y usa `typeof` para obtener su tipo.
3. Usa `as const` en un array de colores y obtén el tipo de un elemento con `typeof arr[number]`.

4. Usa `satisfies` para verificar que un objeto cumple un tipo pero manteniendo el tipo inferido.

7. Generics (Genéricos)

 **Ejercicio:** `s03/02-generics.ts`.  **Solución:** `soluciones/s03/02-generics.ts`.  Prioridad **P1** — `useState<T>`, `invoke<T>` y componentes `<T>` (p. ej. tabla genérica).

1. Escribe la función de identidad con genérico: `identidad<T>(valor: T): T` que devuelva el valor sin cambiar de tipo.
2. Escribe `primero<T>(arr: T[]): T | undefined` que devuelva el primer elemento o `undefined` si el array está vacío.
3. Escribe un filtro genérico `filtrarPor<T>(arr, predicado)` con un predicado tipado.
4. Usa `extends` para restringir un genérico a tipos con `.length` (o con una propiedad `id`).
5. Implementa una cola genérica `crearCola<T>()` con `encolar`, `desencolar` y `estaVacía` — como objeto devuelto por una factoría (sin clases).
6. Escribe un acceso indexado seguro: `obtenerValor<T, K extends keyof T>(obj, key)`.

8. Utility Types

 **Ejercicio:** `s03/03-utility-types.ts`.  **Solución:** `soluciones/s03/03-utility-types.ts`.  Prioridad **P3** — `Partial`/`Pick`/`Omit`/`Record` en formularios de edición y props.

1. Dada una interface `Producto { id: number; nombre: string; precio: number; descripcion?: string }`:
 - Crea un tipo `ProductoParcial` con todas las propiedades opcionales.
 - Crea un tipo `ProductoSinDescripcion` que omita `descripcion`.
 - Crea un tipo `ResumenProducto` que solo tenga `id` y `nombre`.
2. Usa `Record` para crear un tipo `Semana` con días como claves y `string` como valores.
3. Usa `Parameters` y `ReturnType` con una función existente.
4. Usa `NonNullable` para eliminar `null` | `undefined` de un tipo unión.

9. Módulos en TypeScript

 **Ejercicio:** `s03/05-modulos/`.  **Solución:** `soluciones/s03/05-modulos/`.  Prioridad **P4** — se aprende con los repos; `import type` al tipar `invoke`.

1. Crea un módulo `utilidades.ts` con funciones `saludar`, `despedir` y una constante `PI`. Impórtalas en `app.ts`.
2. Exporta una interface desde un módulo y usa `import type`.
3. Crea un módulo con un `export default class` y otro con re-exportaciones.
4. ¿Qué diferencia hay entre `export { x }` y `export default x`?

10. Strict Mode y Configuración

 Configuración ya activa en `ejercicios/tsconfig.json` (`strict: true`, `noUncheckedIndexedAccess`, etc.). **No hay ejercicio dedicado:** es teoría de configuración; el apunte de referencia es `apuntes/s03/10-NPM.md`.

1. Activa `strict: true` en un `tsconfig.json` y comprueba qué flags se activan.
2. Declara una propiedad de clase sin inicializar — ¿qué error da `strictPropertyInitialization`?

3. Usa **!** (definite assignment assertion) para solucionar el error anterior.
4. ¿Qué hace **noImplicitAny**? Escribe una función sin tipo en el parámetro y observa el error.

11. Programación asíncrona (promesas y async/await)

 **Ejercicio:** [s03/08-async.ts](#)  **Solución:** [soluciones/s03/08-async.ts](#)  Prioridad **P1** — **Promise.all + try/catch** = patrón **invoke** y del **useEffect** de carga.

1. Crea una función **esperar(ms)** que devuelva una promesa que se resuelva tras **ms** milisegundos.
2. Escribe una función **async cargarDatos** que devuelva una lista tras un pequeño retardo.
3. Lanza dos promesas a la vez con **Promise.all** en **cargarParalelo**.
4. Usa **try/catch** en **procesarSeguro** para nunca propagar errores.
5. Escribe **hacerTarea(callback: () => void)** que imprima "inicio", ejecute el **callback** y luego imprima "fin". Llámala pasando un **callback** que imprima "tarea".
6. Escribe **mapearConCallback(numeros: number[], callback: (n: number) => number): number[]** que devuelva un array nuevo aplicando el **callback** a cada elemento (sin usar **map**).
7. Usa **setTimeout** para simular una operación asíncrona: **simularEspera(ms, callback)** que llame al **callback** tras **ms** milisegundos. Ordena los **console.log** para comprobar que el callback se ejecuta después.

12. Desestructuración, spread/rest y optional chaining (puente a React)

 **Puente a React.** Es lo primero que usarás en **cada** componente: destructuring de **props** en la firma de la función, **const [valor, setValor] = useState(...), setState({ ...prev, ... })** y **?./??** para navegar datos anidados (API, store). **Prioridad máxima antes de S04.**  **Ejercicio:** [s03/09-desestructuracion-spread-optional.ts](#).  **Solución:** [soluciones/s03/09-desestructuracion-spread-optional.ts](#).  Prioridad **P1** — destructuring de props, spread de **setState**, **?./??**: lo primero de cada componente.

1. Crea **Tarjeta({ nombre, edad, activo })** que reciba el objeto como parámetro y lo desestructure en la firma (patrón **props**). Devuelve el nombre, la edad y si está en línea.
2. Desestructura la tupla **[valor, setValor]** que devuelve una factoría **useMiniEstado** (patrón **useState**). Escribe **incrementarContador** que sume 1.
3. Implementa **resumenPerfil(resp)** con desestructuración **anidada** de **resp.datos.usuario**, extrayendo **nombre**, **perfil.ciudad** (default "desconocida") y **perfil.bio** (default "Sin bio") en una sola línea.
4. Usa **rest** en destructuring: **separarId(pelicula)** debe separar **id** y devolver el resto (el patrón de **<Componente {...resto} />**).
5. Usa **spread** de objetos: **mergeConfig(parcial)** mezcla **CONFIG_DEFECTO** con **parcial** en un objeto nuevo sin mutar (patrón **setState({ ...prev, ...parcial })**).
6. Usa **spread** de arrays: **anadirPuntuacion(arr, nueva)** añade sin mutar (alternativa inmutable a **push**) y **mejorPuntuacion(...args)** despliega un array en **Math.max**.
7. Usa **optional chaining** **?. + ??**: **direccionEnvio(carrito)** navega **carrito.cliente.envio.direccion** (todo opcional) con default "Sin dirección".
8. Desestructura en el parámetro con **valores por defecto**: **Boton({ etiqueta = "Enviar", deshabilitado = false })**.
9. Implementa el **update funcional**: **acumularConUpdater()** con un setter que reciba **(prev) => prev + ...** y encadena tres actualizaciones (**+1, +2, +3**) — es el **setCuenta((c) => c + 1)** de React, que

evita clausuras obsoletas.